Series 1 – Time complexity measurement

Exercise 1

Let A be a one-dimension array containing **n** integers. We want to delete a given value(s) from this array. Two methods to perform this task, the first one using nested loops and the second one using a single loop.
In this exercise, we have to delete all values that are less or equal than 50.

1- Creation of the dataset:

Create two identical array, A1 an A2, with large size (e.g. $n \geq 200000$), and fill them with random values in the range of 0 to100.

2- Processing:

Write the block deletion actions for each method using a single C program.

Using 'clock()' function from the 'time.h' library, measure the running time of the first method using array A1.

- Record the starting time just before the block of deletion actions.
- Perform the actions.
- Record the ending time just after the block of deletion actions.
- Calculate the elapsed time in seconds by subtracting the start time from the end time and dividing by the constant CLOCKS_PER_SEC.
- Print the elapsed time.

Repeat the same steps with the second method using the array A2.
Compare and comment the results.

Exercise 2

Let A be a one-dimension array containing **n** integers. We want to determine whether a given value exists in this array. In this exercise, we have to compare three search algorithms: *sequential search*, *indexed sequential search* and *binary search*. Note that the two last methods can only be applied to sorted arrays.

1- Creation of the dataset:

The asymptotic complexity of these three algorithms are respectively O(n), O(n) and O(Log(n)). However, to be able to have a significant running time we must use an array of very large size (e.g. $\geq 10^9$). Such size can't be used with static array, so we have to use dynamic array.

The array must be sorted, but sorting an array with such size will take a huge time, therefore we use the following trick to create the array: Fill the array with even values, where the value of each cell is equal to the double of its index.

2- Processing:

Write the block search actions for each method using a single C program.

Input the value to search for: we can use different cases:

- The value doesn't exist, you have just to give an odd value.
- The value exists, you have just to input an even value in the range of 0 to 2*size.

- You can also check the three cases (the front, the middle or the end of the array) by providing a value for each case.

To calculate the running time:

- Record the starting time just before the block search actions.
- Perform the actions.
- Record the ending time just after the block of deletion actions.
- Calculate the elapsed time in seconds by subtracting the start time from the end time and dividing by the constant CLOCKS_PER_SEC.
- Print the elapsed time.

Repeat the same steps with each of the different methods using the same array A.
Compare and comment the results.